

TRABALHO N1 – TESTE DE SOFTWARE

Alunos: João Pedro Souza Pereira, Kenzo Matsunaga, Giulia, Kaylane

Introdução ao sistema desenvolvido

O sistema desenvolvido se trata de um CRUD com o tema: catálogo de filmes. Para que todos os testes (testes unitários, testes de API e testes E2E) possam ser realizados, decidimos criar uma API Rest utilizando o Java 17 com o framework do Spring Boot. Nossa API também integra a API da OMDB, que fornece dados sobre diversos filmes e séries.

Decidimos escolher o Java 17 por ser uma versão atual, mas ao mesmo tempo já consolidada no meio do desenvolvimento, já que versões mais novas como o JDK 21 ou JDK 24 são muito recentes e podem conter instabilidades. Também optamos pelo spring boot por ser o principal framework conhecido no meio de back end em Java, por facilitar o desenvolvimento de uma API, utilizando anotações como @RestController @RequestMapping e etc. Por último, decidimos utilizar a API da OMDB por ser fornecida gratuitamente, salvo o limite de 1000 utilizações diárias.

Nosso sistema tem as funcionalidades de um CRUD comum, com operações como: POST, GET, PUT e DELETE caso fossem traduzidas para requisições HTTP.

FUNÇÃO 1 – CADASTRAR FILME: permite cadastrar um novo filme na aplicação. Exemplo de filme:

```
{ "titulo": "Matrix", "genero": "Ficcao", "anoLancamento": 1999, "duracao": 136 }
```

Quando o cadastro dá certo, a API retorna 201 created. O sistema também valida os dados. Ele não aceita título vazio, gênero vazio, filme duplicado com mesmo título e mesmo ano, e etc.

FUNÇÃO 2 – LISTAR FILME: permite listar todos os filmes cadastrados com a rota: GET/filmes.

Se o filme existir, ele retorna uma lista com os filmes salvos no banco, como por exemplo:

```
{ "id": 1, "titulo": "Matrix", "genero": "Ficcao", "anoLancamento": 1999, "duracao": 136 }
```

FUNÇÃO 3 – BUSCAR FILME POR ID: permite buscar um filme específico pelo seu id pela rota (GET/filmes/1).

Caso o filme exista, o sistema retorna os dados dele, e se não existir, retorna erro o 404 not found.

FUNÇÃO 4 – ALTERAR FILME: permite alterar os dados de um filme já cadastrado pela rota (PUT/filmes/1), como por exemplo:

```
{ "titulo": "Matrix Reloaded", "genero": "Acao", "anoLancamento": 2003, "duracao": 138 }
```

Se o filme existir e os dados forem válidos, ele atualiza e retorna o código 200 ok, mas se o filme não existir, retorna 404. Caso os dados forem inválidos, ele retorna 400. Por último, se tentar alterar para um título e ano que já existem em outro filme, ele retorna 409 Conflict.

FUNÇÃO 5 – EXCLUIR FILME: permite excluir um filme pelo id pela rota (DELETE/filmes/1). Se excluir com sucesso, o sistema retorna 204 no conteúdo, mas se o filme não existir, ele retorna 404 not found.

FUNÇÃO 6 – CONSULTAR FILMES NA OMDb: Além do cadastro manual, o sistema consegue consultar dados de filmes da API OMDb pela rota (GET/filmes/omdb?titulo=*nome*&ano=2010). O campo ano é opcional.

Essa função busca o filme no OMDb, mas não salva no banco, ela só mostra os dados encontrados.

Exemplo de uso:

URL: GET /filmes/omdb?titulo=Inception

Pode retornar algo como:

```
{"titulo": "Inception", "genero": "Action, Adventure, Sci-Fi", "anoLancamento": 2010, "duracao": 148}
```

FUNÇÃO 7 – IMPORTAR FILMES DA OMDb: Essa funcionalidade busca um filme no OMDb e salva no catálogo automaticamente pela rota (POST /filmes/importar)

Um adendo é que o método GET só consulta, não salva e o método POST consulta e salva no banco.

FUNÇÃO 8 – TRATAMENTO DE ERROS: O projeto tem respostas de erro organizadas. Por exemplo, se tentar cadastrar um filme sem título, a API retorna erro e diz qual é o erro, como por exemplo: "O título é obrigatório!". Isso ajuda nos testes porque fica fácil verificar se o sistema respondeu corretamente.

FUNÇÃO 9 – INTERFACE CLI: O projeto também tem uma interface de linha de comando. Ela permite fazer as mesmas operações principais, como listar filmes, buscar filme, cadastrar, alterar, excluir, consultar no OMDb e importar do OMDb.

Planejamento de testes

O planejamento de testes do sistema será baseado nas principais funcionalidades do catálogo de filmes. O objetivo será confirmar se o sistema faz corretamente as operações de cadastro, pesquisa, mudança, exclusão e importação de filmes, além de validar se os erros vão ser tratados de forma adequada quando o usuário informa dados inválidos ou nulos.

Pensando nisso, os testes serão divididos em três categorias: testes unitários, testes de API e testes E2E. Essa divisão vai nos possibilitar avaliar o sistema em níveis diversos, desde métodos isolados até o funcionamento completo simulando o uso por um usuário.

Os testes unitários serão usados para validar as funções e regras individuais da aplicação. Eles focarão principalmente na parte responsável pelas regras de negócio, verificando ocasiões como: cadastro de filme válido, cadastro (mas com campos obrigatórios vazios), tentativa de cadastro duplicado, busca de um filme que já existe, busca de filme inexistente, alteração de dados e deletar um filme. Esses testes vão ser importantes para garantir que cada regra funcione de forma isolada.

Os testes de API serão utilizados para validar os endpoints que são responsáveis pelas operações do catálogo. Nós simularemos as requisições HTTP para rotas de POST, GET, GET por id, PUT por id, DELETE por id e POST. Teremos como objetivo verificar se a API retorna os dados que são esperados e os códigos HTTP corretos, como 201 created, 200 ok, 204 no content, 400 bad request, 404 not found e etc.

Os testes E2E vão ser usados para validar o funcionamento completo do sistema com a interface, simulando a utilização por um usuário. Esses testes vão executar fluxos completos, como cadastrar um filme pela interface, consultar o filme cadastrado, alterar seus dados e realizar a deletar. Esse tipo de teste verifica se todas as partes do sistema funcionam corretamente em conjunto, desde a interação do usuário até o processamento e retorno das informações.

A estratégia de testes também incluirá cenários positivos e negativos. Os cenários positivos veem se o sistema funciona corretamente quando os dados informados são válidos. Por fim, os cenários negativos vão verificar se o sistema identifica erros, como campos obrigatórios vazios, filme inexistente, dados inválidos ou tentativa de cadastro duplicado.

Além disso, poderão ser criados testes com falhas propositas, com o objetivo de demonstrar a efetividade dos testes. Esses testes terão resultados esperados incorretos de forma intencional, permitindo comprovar que as ferramentas de teste conseguem identificar comportamentos divergentes.

Dessa forma, o planejamento busca garantir que as principais funcionalidades do catálogo de filmes sejam avaliadas em diferentes níveis, aumentando a confiabilidade do sistema e permitindo identificar falhas tanto nas regras internas quanto na comunicação da API e no fluxo completo de uso pelo usuário.

Descrição dos casos de teste

A descrição dos casos de teste apresenta os cenários que serão utilizados para verificar se o sistema funciona corretamente. Nessa etapa, são definidos o objetivo de cada teste, o tipo de teste realizado, a entrada ou situação analisada e o resultado esperado. Dessa forma, é possível organizar melhor a validação das funcionalidades do sistema e identificar se ele atende ao comportamento esperado.

Código	Tipo	Objetivo	Entrada ou cenário	Resultado esperado
CT01	Unitário	Validar cadastro de filme valido	Filme com titulo, gênero, ano e duração validos	O filme deve ser cadastrado com sucesso
CT02	Unitário	Validar campos obrigatórios	Filme sem titulo ou sem gênero	O sistema deve retornar erro de dados inválidos
CT03	Unitário	Validar filme duplicado	Cadastro com mesmo titulo e mesmo ano de um filme existente	O sistema deve impedir o cadastro
CT04	Unitário	Validar busca de filme inexistente	Busca por um ID que não esta cadastrado	O sistema deve retornar erro de filme não encontrado
CT05	API	Cadastrar filme pela API	Requisição POST /filmes com JSON valido	A API deve retornar 201 Created
CT06	API	Rejeitar cadastro invalido pela API	Requisição POST /filmes com campos inválidos	A API deve retornar 400 Bad Request
CT07	API	Listar filmes cadastrados	Requisição GET /filmes	A API deve retornar 200 OK e a lista de filmes
CT08	API	Alterar filme pela API	Requisição PUT /filmes/{id} com dados validos	A API deve retornar 200 OK e os dados atualizados
CT09	API	Excluir filme pela API	Requisição DELETE /filmes/{id} com ID existente	A API deve retornar 204 No Content
CT10	E2E	Validar fluxo completo de cadastro e consulta	Usuário cadastra um filme e depois consulta a listagem	O filme cadastrado deve aparecer na lista
CT11	E2E	Validar fluxo completo de alteração	Usuário cadastra um filme e altera seus dados	Os novos dados devem ser exibidos corretamente
CT12	E2E	Validar fluxo completo de exclusão	Usuário cadastra um filme e depois exclui	O filme nao deve mais aparecer na listagem